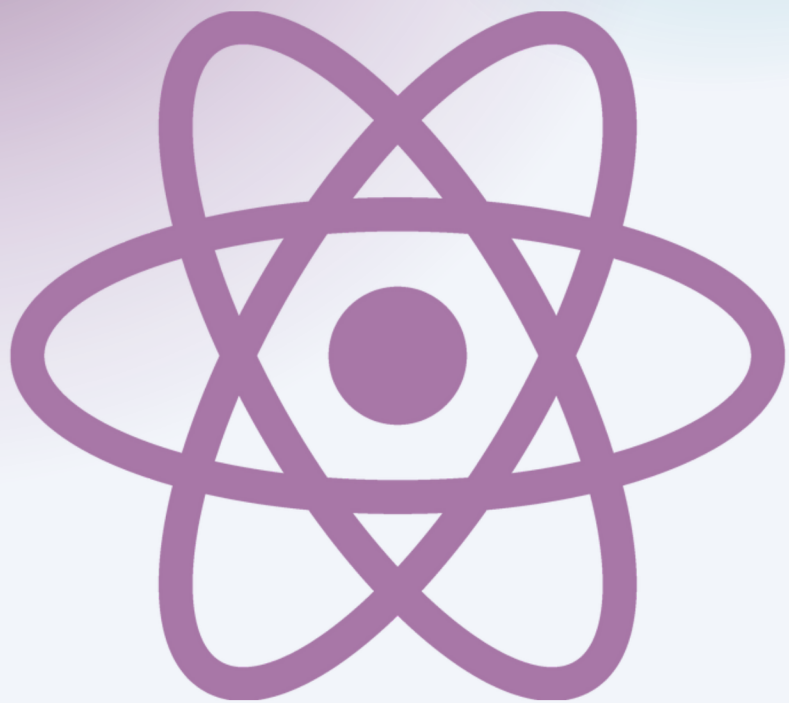




REACT CUSTOM HOOKS

**4 Useful Custom Hooks for
your next Project**

SWIPE



useLocalStorage

Easily store and retrieve data from local storage.

```
import { useState } from 'react';

function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {  
    const storedValue = localStorage.getItem(key);  
    return storedValue ? JSON.parse(storedValue) : initialValue;  
  });

  const setStoredValue = (newValue) => {  
    setValue(newValue);  
    localStorage.setItem(key, JSON.stringify(newValue));  
  };

  return [value, setStoredValue];
}

// Usage:  
// const [name, setName] = useLocalStorage('name', 'John');
```

useMediaQuery

Detect & add to viewport size changes

```
import { useState, useEffect } from 'react';

function useMediaQuery(query) {
  const [matches, setMatches] = useState(false);

  useEffect(() => {
    const mediaQuery = window.matchMedia(query);
    setMatches(mediaQuery.matches);

    const handler = (e) => setMatches(e.matches);
    mediaQuery.addEventListener('change', handler);

    return () => {
      mediaQuery.removeEventListener('change', handler);
    };
  }, [query]);

  return matches;
}

// Usage:
// const isMobile = useMediaQuery('(max-width: 768px)');
```

useDebounce

Debounce user inputs or anything that triggers more than once and needs throttling. Useful on search inputs.

```
import { useState, useEffect } from 'react';

function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

// Usage:
// const debouncedSearchTerm = useDebounce(searchTerm, 300);
```

useFetch

Simplify & standardize fetching data from an external API.

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchData() {
      try {
        const response = await fetch(url);
        if (!response.ok) {
          throw new Error('Network response was not ok');
        }
        const json = await response.json();
        setData(json);
      } catch (err) {
        setError(err);
      } finally {
        setIsLoading(false);
      }
    }

    fetchData();
  }, [url]);

  return { data, isLoading, error };
}

// Usage:
// const { data, isLoading, error } = useFetch('https://api.examp
```

LIKE
&
FOLLOW
fore more content



Mario Ruci

<https://marioruci.com>